

第30回 実践：Web ゲームプログラミング 9

実習時のフォルダ構成

Web_名前 の中に 30 フォルダを作る。第 29 回の sample フォルダと exercise フォルダを、30 の中にコピーして、続きから作る。

```
Web_名前
├── 30
│   ├── sample          ← 第 29 回の sample をコピーする（今回は書き足さない）
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise        ← 第 29 回の exercise をコピーする
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- **sample**：講義で使う完成版のシューティングゲーム（第 28 回で完成した **sample** と同じ）
- **exercise**：自分のシューティングゲーム。続きから作る。

sample/index.html の内容は、次のとおり。<canvas> の id と大きさ（幅 400、高さ 500）を確認する。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>シューティングゲーム</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <canvas id="game" width="400" height="500"></canvas>
  <script src="main.js"></script>
</body>
</html>
```

講義内容

今回は最終回。これまでに作ってきたシューティングゲームを、次の「最終必須仕様」をすべて満たす形に仕上げる。

1. 最終必須仕様

1. Canvas に自機を表示
2. キーで左右移動
3. キーで弾を発射
4. 敵が出現して移動

5. 弾が敵に当たる
6. 当たると敵が消えて得点が増える
7. 敵などとの接触で残機が減る
8. 残機 0 でゲームオーバー
9. リスタートできる
10. それぞれの仕様を学んだ回と、`sample/main.js` で担当している関数は、第 29 回の講義内容 1 の表で確認する。

`sample/index.html` をブラウザで開いて、操作を確認する。

キー	動作
← →	自機を左右に動かす
スペース	弾を撃つ（押し続けると連射）
Enter	ゲームオーバーのあとに、リスタートする

2. プログラムの全体の流れ

ゲームは、`gameLoop` を 1 秒間に約 60 回くり返して動いている。1 回ごとに「更新（`update`）→ 描画（`draw`）」を行う。

```
gameLoop
├─ プレイ中なら update（動かす・当たり判定）
│   ├─ updatePlayer      自機を動かす
│   ├─ updateBullets     弾を撃つ・動かす・画面外で消す
│   ├─ updateEnemies     敵を出す・動かす・画面外で消す
│   ├─ checkBulletHits   弾と敵の当たり判定
│   └─ checkPlayerHit    敵と自機の当たり判定
├─ draw（描く）
│   └─ 背景 → 自機 → 弾 → 敵 → 得点・残機（→ ゲームオーバーの文字）
└─ requestAnimationFrame(gameLoop)  次の回を予約する
```

- `gameState` が “play” の間だけ `update` を行う。“gameover” になると、動きが止まり、描画だけが続く。

3. 完成前の確認

うまく動かないときは、次の表で確認場所を探す。コンソールのエラーの読み方と調べ方は、第 29 回の講義内容 2 を見る。

症状	確認すること
自機が動かない	<code>event.key</code> の綴り（ <code>"ArrowLeft"</code> 、 <code>"ArrowRight"</code> ）。大文字・小文字も同じか
自機が画面の外に出る	画面の端で止める <code>if</code> 文があるか。右端は <code>player.w</code> を足して比べているか
弾・敵が出ない	<code>push</code> しているか。 <code>draw</code> から <code>drawBullets</code> ・ <code>drawEnemies</code> を呼び出しているか
当たっても敵が消えない	<code>update</code> から <code>checkBulletHits</code> を呼び出しているか。 <code>isHit</code> の4つの条件
リスタート後に敵や得点が残る	<code>resetGame</code> で、配列を <code>[]</code> に、得点・残機を最初の値に戻しているか

4. 参考：完成版のコード

`sample/main.js` の内容を、上から順に載せる。つなげると、`sample/main.js` 全体になる。講義では全部は読まない。演習で、自分のコードと見比べるときに使う。

(1) 準備と変数（仕様 1）

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// ===== ゲームの状態 =====
let gameState = "play";           // "play": プレイ中  "gameover": ゲームオーバー
let score = 0;
let lives = 3;
let frameCount = 0;               // ゲーム開始からのフレーム数

// ===== 自機・弾・敵 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
let bullets = [];
let enemies = [];
let shotTimer = 0;                // 次の弾を撃てるまでのフレーム数
```

- 得点（`score`）、残機（`lives`）、状態（`gameState`）は、ゲーム全体で使うので、ファイルの先頭で用意する。
- 自機・弾・敵は、`{ x, y, w, h }`（位置と大きさ）を持つオブジェクトで表す。弾と敵は数が変わるので、配列に入れる。

(2) キー入力（仕様 2・3・9）

```
// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;
let isSpacePressed = false;

document.addEventListener("keydown", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
  if (event.key === " ") {
    isSpacePressed = true;
  }
  if (event.key === "Enter" && gameState === "gameover") {
    resetGame();
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
  if (event.key === " ") {
    isSpacePressed = false;
  }
});
```

- キーを押したときに `true`、離したときに `false` にしておき、実際の移動や発射は `update` の中で行う。
- `Enter` キーは、ゲームオーバーのときだけリスタートする（`&&` で 2 つの条件をつなげている）。

(3) リスタート（仕様 9）

```
// ===== リスタート =====
function resetGame() {
  gameState = "play";
  score = 0;
  lives = 3;
  frameCount = 0;
  player.x = 184;
  bullets = [];
  enemies = [];
  shotTimer = 0;
}
```

- ゲームの途中で変わる変数を、**すべて**最初の値に戻す。戻し忘れると、リスタート後に前のゲームの敵や得点が残る。

- 最初の値は、ファイルの先頭と `resetGame` の 2 か所を書いてある。値を変えるときは、両方を変える。

(4) 自機・弾・敵を動かす（仕様 2・3・4）

```
// ===== 更新 =====
function updatePlayer() {
  if (isLeftPressed) {
    player.x -= player.speed;
  }
  if (isRightPressed) {
    player.x += player.speed;
  }
  // 画面の端で止める
  if (player.x < 0) {
    player.x = 0;
  }
  if (player.x + player.w > canvas.width) {
    player.x = canvas.width - player.w;
  }
}
```

- 第 18 回で学んだ「画面の端で止める」処理。右端は、`player.x + player.w`（自機の右端）で比べる。

```
function updateBullets() {
  if (shotTimer > 0) {
    shotTimer -= 1;
  }
  // スペースキーを押していて、撃てる状態なら、弾を追加する
  if (isSpacePressed && shotTimer === 0) {
    bullets.push({ x: player.x + player.w / 2 - 2, y: player.y, w: 4, h: 12, speed: 8 });
    shotTimer = 10;
  }
  for (let i = bullets.length - 1; i >= 0; i--) {
    bullets[i].y -= bullets[i].speed;
    // 画面の上に出た弾を消す
    if (bullets[i].y + bullets[i].h < 0) {
      bullets.splice(i, 1);
    }
  }
}
```

- `shotTimer` は、次の弾を撃てるまでの待ち時間。弾を撃つと 10 になり、1 フレームごとに 1 ずつ減る。0 になると、また撃てる。
- 弾の `x` は、自機の中央（`player.x + player.w / 2`）から、弾の幅の半分（2）を引いた位置にする。

```
function updateEnemies() {
  // 60 フレーム（約 1 秒）ごとに、敵を 1 体出す
  if (frameCount % 60 === 0) {
    enemies.push({ x: Math.floor(Math.random() * (canvas.width - 32)), y: -32, w: 32, h: 32, speed: 2 });
  }
  for (let i = enemies.length - 1; i >= 0; i--) {
    enemies[i].y += enemies[i].speed;
    // 画面の下に出た敵を消す
    if (enemies[i].y > canvas.height) {
      enemies.splice(i, 1);
    }
  }
}
}
```

- `frameCount % 60 === 0` は、「フレーム数が 60 で割り切れるとき」、つまり 60 フレームに 1 回。
- 出現位置の `x` は、第 18 回の「`Math.floor(Math.random() * 個数) + 最小値`」の形。敵が画面の右からはみ出さないように、`canvas.width - 32`（敵の幅を引いた値）を使う。
- 配列から `splice` で消すときは、配列の**後ろから**順に調べる。前から調べると、消した後ろの要素が 1 つ前に詰まり、調べずに飛ばしてしまう要素が出る。

(5) 当たり判定（仕様 5・6・7・8）

```
// 2 つの四角形が重なっていたら true を返す
function isHit(a, b) {
  return a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y;
}

// 弾と敵の当たり判定
function checkBulletHits() {
  for (let i = bullets.length - 1; i >= 0; i--) {
    for (let j = enemies.length - 1; j >= 0; j--) {
      if (isHit(bullets[i], enemies[j])) {
        bullets.splice(i, 1);
        enemies.splice(j, 1);
        score += 100;
        break; // この弾は消えたので、次の弾へ
      }
    }
  }
}
}
```

- `isHit` は、2 つの四角形が重なっているかを調べる。4 つの条件が**すべて true** のとき、重なっている。
- 当たったら、弾と敵を両方とも配列から消し、得点を増やす。

```
// 敵と自機の当たり判定
function checkPlayerHit() {
  for (let i = enemies.length - 1; i >= 0; i--) {
    if (isHit(player, enemies[i])) {
      enemies.splice(i, 1);
      lives -= 1;
      if (lives <= 0) {
        gameState = "gameover";
      }
      break; // 1 フレームで減る残機は 1 つまで
    }
  }
}
```

- 敵に当たったら、その敵を消して、残機を 1 減らす。残機が 0 以下になったら、`gameState` を `"gameover"` にする。

```
function update() {
  frameCount += 1;
  updatePlayer();
  updateBullets();
  updateEnemies();
  checkBulletHits();
  checkPlayerHit();
}
```

- `update` は、更新の処理をまとめて呼び出す。当たり判定は、全員を動かしたあとに行う。

(6) 描画 (仕様 1・6・7・8)

```
// ===== 描画 =====
function drawBackground() {
  ctx.fillStyle = "#000022";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

function drawBullets() {
  ctx.fillStyle = "#ffff00";
  for (let i = 0; i < bullets.length; i++) {
    ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);
  }
}

function drawEnemies() {
  ctx.fillStyle = "#ff4444";
  for (let i = 0; i < enemies.length; i++) {
    ctx.fillRect(enemies[i].x, enemies[i].y, enemies[i].w, enemies[i].h);
  }
}
```

```

function drawUI() {
  ctx.fillStyle = "#ffffff";
  ctx.font = "20px sans-serif";
  ctx.textAlign = "left";
  ctx.fillText(`SCORE: ${score}`, 10, 30);
  ctx.fillText(`LIVES: ${lives}`, 10, 60);
}

function drawGameOver() {
  ctx.fillStyle = "#ffffff";
  ctx.textAlign = "center";
  ctx.font = "40px sans-serif";
  ctx.fillText("GAME OVER", canvas.width / 2, 230);
  ctx.font = "20px sans-serif";
  ctx.fillText("Enter キーでリスタート", canvas.width / 2, 280);
}

function draw() {
  drawBackground();
  drawPlayer();
  drawBullets();
  drawEnemies();
  drawUI();
  if (gameState === "gameover") {
    drawGameOver();
  }
}

```

- 描いた順に上へ重なる。最初に背景を描き、文字は最後に描く。
- 得点・残機は、テンプレートリテラルで `fillText` に渡している。

(7) ゲームループ

```

// ===== ゲームループ =====
function gameLoop() {
  if (gameState === "play") {
    update();
  }
  draw();
  requestAnimationFrame(gameLoop);
}

gameLoop();

```

- 最後の `gameLoop();` で、ゲームが始まる。

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。結果は、`exercise/index.html` をブラウザで開いて確認する。変数名や関数名が `sample` と違うときは、自分のゲームに合わせて読み替える。

演習 1 最終必須仕様を満たす

- 自分のゲームで、最終必須仕様 1~9 を 1 つずつ確認する。
- できていない仕様は、講義内容と `sample/main.js` を参考にして追加する。
- 最後に、次をすべて確認する。
- ゲームを始めてから、ゲームオーバーになるまで、コンソールにエラーが出ない。
- ゲームオーバーのあとにリスタートすると、得点・残機・自機の位置が最初に戻り、弾と敵が消えている。

演習 2 自分のゲームに調整する

- 次の中から 3 つ以上を選んで、値を変え、遊びやすさ（難しさ）を自分で調整する。見た目（色・形）は、第 29 回の演習 3・発展で調整した。

調整する値	<code>sample/main.js</code> での場所
自機の速さ	<code>player</code> の <code>speed</code>
弾の間隔	<code>updateBullets</code> の <code>shotTimer = 10;</code>
弾の速さ	<code>updateBullets</code> の弾の <code>speed</code>
敵の出現間隔	<code>updateEnemies</code> の <code>frameCount % 60</code>
敵の速さ	<code>updateEnemies</code> の敵の <code>speed</code>
1 体倒したときの得点	<code>checkBulletHits</code> の <code>score += 100;</code>
残機の数	先頭の <code>let lives = 3;</code> と <code>resetGame</code>

- `exercise/main.js` の先頭に、どの値を何から何に変えたかを、コメントで書く。

発展（任意）

- 難易度**：得点が増えるほど、敵の出現間隔が短く、敵が速くなるようにする。（ヒント：60 や 2 を変数にして、得点から計算する。第 18 回の「上限・下限で止める」if 文で、速くなりすぎないようにする。）
- 無敵時間と点減**：残機が減ったあと、しばらくは敵に当たっても残機が減らないようにする。その間、自機を点減させる。（ヒント：% を使うと、「表示する・しない」を交互に切り替えられる。）
- タイトル画面**：`gameState` に “title” を加え、最初にタイトルと操作方法を表示して、Enter キーでゲームを始める。
- ハイスコア**：リスタートしても消えない最高得点を用意し、画面に表示する。